Guide to Debugging Bash Scripts

1. Debugging Bash Scripts Natively

Bash provides several built-in options for debugging scripts. Here's how to use them effectively:

1. Enable Debugging Flags:

   - Use the `-x` option to print each command and its arguments as they are executed:

     Example: `bash -x ./script.sh`

   - Add the `-x` flag to the shebang line in the script:

     Example: `#!/bin/bash -x`

2. Verbose Mode:

   - Use the `-v` option to print each command before execution:

     Example: `bash -v ./script.sh`

3. Combine Debugging Flags:

   - Combine `-x` and `-v` for detailed debugging:

     Example: `bash -xv ./script.sh`

4. Use the `set` Command:

   - Enable and disable debugging dynamically within the script:

     Example:

     ```bash
     set -x
     # Debugged commands
     ```

set +x

```

5. Use `trap` for Debugging:

- Use `trap` to capture debugging events dynamically.

Example: `trap 'echo "Executing line: $LINENO"' DEBUG`

6. Customize Debug Prompts:

- Set the `PS4` variable for more informative debug output.

Example:

```bash
export PS4='+ (${BASH_SOURCE}:${LINENO}): '
bash -x ./script.sh
```

7. Check Syntax Without Running:

- Use the `-n` option to check syntax errors.

Example: `bash -n ./script.sh`

2. Breakpoints in Bash

Bash does not natively support traditional breakpoints but allows similar functionality:

1. Use the `trap` Command:

- Insert `trap` to pause execution and display debug information.
```

Example:

```bash
trap 'read -p "Breakpoint at $LINENO. Press Enter to continue..."' DEBUG
```

2. Manual Pauses:

- Use the `read` command to pause execution manually.

Example:

```bash
echo "Stopping for manual debug at line $LINENO"
read -p "Press Enter to continue..."
```

3. Debugging Bash Scripts with VSCode

VSCode provides a rich debugging environment for Bash scripts. Here's how to set it up:

1. Install Bash Debugger Extension:

- Install "Bash Debug" from the VSCode marketplace.

2. Configure VSCode Debugging:

- Add the following configuration to `.vscode/launch.json`:

```json
{
    "version": "0.2.0",
```

```
    "configurations": [
        {
            "type": "bashdb",
            "request": "launch",
            "name": "Bash Debug",
            "program": "${file}",
            "cwd": "${workspaceFolder}",
            "pathBash": "/bin/bash",
            "args": [],
            "env": {}
        }
    ]
}
```

3. Using Breakpoints in VSCode:

- Add breakpoints by clicking in the left margin next to line numbers.
- Start debugging from the "Run and Debug" menu.

4. Combining VSCode with Native Debugging:

- You can use `trap` or `read` commands alongside VSCode debugging for manual inspection.

4. Tips for Effective Debugging

- Use `echo` or `printf` to log variable states and outputs.

Guide to Debugging Bash Scripts

- Leverage `ShellCheck` for static analysis to detect potential issues.

- Break scripts into smaller functions for modular debugging.